📋 Таска: Настроить CORS для связи с фронтендом

🎯 Цель

Разрешить браузеру с фронтенда (`http://127.0.0.1:3000`) делать запросы к нашему API (`http://127.0.0.1:8000`). Сейчас при первой же попытке `fetch` фронт получает ошибку `blocked by CORS policy` — это ожидаемо, потому что порты разные, а Django по умолчанию такие запросы не разрешает.

🧠 Что такое CORS (в двух словах)

CORS — это защитный механизм браузера. Когда JS с одного origin (порт `3000`) пытается обратиться к другому origin (порт `8000`), браузер сначала спрашивает у сервера: «Ты разрешаешь этому сайту к тебе ходить?». Если сервер не отвечает специальными заголовками (`Access-Control-Allow-Origin`), браузер блокирует запрос. Мы должны сказать Django, чтобы он эти заголовки добавлял для нашего фронтенда.

📦 Шаг 1: Установка пакета

```
pip install "django-cors-headers==4.9.0"
```

Зафиксируй версию в `requirements.txt` :

```
pip freeze > requirements.txt
```

> ⚠️ Версия `4.9.0` актуальна на момент написания (PyPI). Если выйдет новее — можно обновить, но сначала проверь, что ничего не сломалось.

⚙️ Шаг 2: Настройка `settings.py`

Открой `backend/config/settings.py` и внеси три изменения.

1. Добавь `corsheaders` в `INSTALLED_APPS` :

```
INSTALLED_APPS = [
    # ... твои приложения
    "corsheaders",
]
```

2. Добавь `CorsMiddleware` в `MIDDLEWARE` — как можно выше:

```
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",  # ← ДОЛЖЕН БЫТЬ ЗДЕСЬ
    "django.middleware.security.SecurityMiddleware",
```

```
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    # ... остальные middleware
]
```

> ❗ **Почему именно наверху?** Middleware в Django выполняется сверху вниз. Если `CorsMiddleware` окажется **после** `CommonMiddleware` или `WhiteNoiseMiddleware`, эти middleware успеют сгенерировать ответ раньше, чем CORS-заголовки будут добавлены, и браузер всё равно заблокирует запрос. Ставь его первым или сразу после `SecurityMiddleware`.

3. Добавь разрешённые origin'ы (в конец файла):

```
# CORS: какие сайты могут обращаться к нашему API
CORS_ALLOWED_ORIGINS = [
    "http://127.0.0.1:3000",
    "http://localhost:3000",  # на случай, если фронт откроют через localhost
]
```

> 💡 **Почему два адреса?** Браузер считает `localhost:3000` и `127.0.0.1:3000` **разными origin'ами**, хотя это одна и та же машина. Если указать только один, а фронтенд запустится на другом — CORS снова сработает. Добавь оба, чтобы не зависеть от того, как студент открыл страницу.

4. (Важно для будущих POST/PUT/DELETE) Добавь `CSRF_TRUSTED_ORIGINS`:

```
# CORS разрешает браузеру ЧИТАТЬ ответ.
# CSRF_TRUSTED_ORIGINS разрешает браузеру отправлять НЕБЕЗОПАСНЫЕ запросы (POST, PUT, DELET
CSRF_TRUSTED_ORIGINS = [
    "http://127.0.0.1:3000",
    "http://localhost:3000",
]
```

> ⚠️ **Не путай CORS и CSRF.** Это разные вещи:
>
> - **CORS** — «можно ли JavaScript с этого сайта читать ответы от твоего API».
> - **CSRF** — «можно ли с этого сайта отправлять формы/изменяющие запросы (POST/PUT/DELETE)».
>
> Если сейчас настроить только `CORS_ALLOWED_ORIGINS`, то `GET`-запросы заработают, но первый же `POST` упадёт с `403 CSRF verification failed`. Поэтому `CSRF_TRUSTED_ORIGINS` нужно добавить сразу.

✅ Шаг 3: Проверка

1. **Перезапусти сервер** (Django не подхватывает изменения `MIDDLEWARE` и `INSTALLED_APPS` на лету в некоторых случаях — лучше перезапустить):

```
python manage.py runserver
```

2. **Проверь через curl (из терминала):**

```
curl -I -H "Origin: http://127.0.0.1:3000" http://127.0.0.1:8000/
```

В ответе должны быть заголовки:

```
Access-Control-Allow-Origin: http://127.0.0.1:3000
Vary: Origin
```

Если `Access-Control-Allow-Origin` есть — CORS работает.

3. **Проверь с фронта:** попроси фронтенд-команду выполнить в консоли браузера на `http://127.0.0.1:3000` :

```
fetch('http://127.0.0.1:8000/')
  .then(r => console.log('OK', r.status))
  .catch(e => console.error('FAIL', e));
```

Должно вывести `OK 404` (или `200` , если есть корневой роут). Ошибки `CORS policy` быть не должно. `404` — это нормально, главное, что **запрос дошёл и ответ читается**.

4. **Проверь, что чужой origin блокируется** (защита работает):

```
curl -I -H "Origin: http://evil.com" http://127.0.0.1:8000/
```

Заголовка `Access-Control-Allow-Origin` быть **не должно** (или он не будет равен `http://evil.com`).

🚫 Чего НЕ делать

- **Не используй `CORS_ALLOW_ALL_ORIGINS = True` .** Это разрешает запросы с любого сайта в интернете. Для локальной разработки с двумя известными портами это избыточно и приучает к плохой практике. Если очень нужно временно — используй, но **обязательно** убери перед любым деплоем или показом проекта.
- **Не хардкодь origin'ы в нескольких местах.** Всё должно быть в `settings.py` , в одном блоке.
- **Не путай `http://127.0.0.1:3000` и `http://localhost:3000` .** Это разные origin'ы для браузера. Добавляй оба, если фронт может открываться по-разному.

📝 Критерии готовности (Definition of Done)

- [ ] `django-cors-headers==4.9.0` добавлен в `requirements.txt`
- [ ] `corsheaders` есть в `INSTALLED_APPS`

- [ ] `corsheaders.middleware.CorsMiddleware` стоит **первым** (или сразу после `SecurityMiddleware`) в `MIDDLEWARE`
- [ ] `CORS_ALLOWED_ORIGINS` содержит оба адреса: `http://127.0.0.1:3000` и `http://localhost:3000`
- [ ] `CSRF_TRUSTED_ORIGINS` содержит те же адреса
- [ ] `curl -I -H "Origin: http://127.0.0.1:3000" http://127.0.0.1:8000/` возвращает `Access-Control-Allow-Origin: http://127.0.0.1:3000`
- [ ] `curl -I -H "Origin: http://evil.com" http://127.0.0.1:8000/` **не** возвращает `Access-Control-Allow-Origin` для `evil.com`
- [ ] Фронтенд может сделать `fetch` с `http://127.0.0.1:3000` на `http://127.0.0.1:8000/` без ошибки CORS
- [ ] Изменения закоммичены